Exact Policy-Aware Regression Diagnosis for Precision-Constrained Threshold Selectors

Abstract

Software changes to precision-constrained threshold selectors can preserve selected observations at tested policies while changing them elsewhere in an allowed domain. We study exact policy-aware regression diagnosis on fixed labeled observations. The method specializes established finite-region reasoning to selected-ID equivalence, retains endpoint ownership and rational/binary64 interface semantics, and produces input-bound certificates checked by a separate implementation. We evaluate correctness, controlled testing, historical upstream defects, computational cost and permitted-domain reuse under distinct protocols. The recorded execution passes 42 core tests. Complete checking identifies 183 configured divergent variants and 136 executed nonconforming runs; the thirteen-point grid identifies 157 and 111, respectively. On two documented upstream defects, complete checking adds no affected-workload detection over that grid. A streaming consumer and a Boolean certificate index expose engineering tradeoffs while preserving explicit checking obligations. The evidence supports complete localization and auditable diagnostics for the admitted finite contract. It does not establish a new general verification algorithm, broader real-bug detection superiority, developer productivity or production-policy effectiveness.

Keywords: regression diagnosis; precision constraints; policy equivalence; independently checked certificates; allowed policy domains.

1 Introduction

1.1 Background and Motivation

Precision-constrained threshold selection turns numerical scores into a set of observations for review or intervention. For example, an alerting component in decision-support software might allow several precision targets as operating conditions change. Software changes must then be assessed against those permitted policies. This scenario motivates the study; the evaluation does not assess a deployed alerting service or an entire cold-chain information system.

We compare selected observation IDs under an agreed empirical contract: labels and aligned IDs are fixed, candidates come from a finite family of score thresholds, and feasibility, priority and abstention are explicit. The question is equality throughout the caller's permitted policy set and, when equality fails, its exact location and a checkable observation witness.

A regression test at one policy establishes compatibility only at that policy. A finite grid gives broader observations, but cannot certify all policies in an interval unless its coverage is justified by the selector contract. The engineering question is consequently more specific than whether a version difference exists: which permitted policies change selected IDs, who owns each boundary, and what evidence allows another implementation to check the diagnosis? A zero-length divergent set can still contain a meaningful endpoint, so an interval-length statistic alone cannot answer the equivalence question.

1.2 Research Gap and Related Work

Rath and Hughes [1], Mackey et al. [2] and Tasche [3] establish constrained threshold-selection foundations; scikit-learn documents the calibration [5] and public selector [21] interfaces used here. Metamorphic/mutation testing of optimization and classifier implementations [4,11,12] provides the testing context. The inherited candidate-frontier manuscript [29] is related prior work, with its reused contract, data and witness machinery identified in Section 1.

Storm and parameter synthesis [6,7,20], regression verification [8], certifying computations [9,17], proof-carrying code [10], robust classification [18] and parametric monitoring [19] reason about related properties or evidence boundaries. Predicate enumeration and AllSMT [23–25] already support general solution-region exploration; rounding-preimage reasoning [26] underlies the numerical-interface mapping. These are positive foundations, not

claims that existing tools lack endpoint or full-region capabilities. Our empirical SMT baselines encode the selected-ID contract directly; they do not reproduce an independently optimized AllSMT implementation.

These foundations already permit parameter-region exploration, constrained selection and producer/checker separation. The gap addressed here is an explicit diagnostic workflow for this finite empirical selector: aligned selected IDs, complete policy cells, endpoint ownership, numerical-interface contracts and an independently consumable record must agree. We do not claim that finite partitioning or general verification was previously unavailable. The empirical question is whether this specialization yields correct and inspectable diagnoses, under which integration assumptions, and at what measured computational cost.

1.3 Research Objectives and Contributions

Our objective is to diagnose selected-ID regressions over a declared policy domain and make the diagnosis checkable from the original inputs. The contribution comprises exact divergent cells with observation witnesses and numerical-interface semantics; an input-bound certificate with separately implemented consumers; and permitted-domain queries evaluated alongside explicit testing and cost baselines. The streaming consumer and Boolean interval index apply established data structures within this contract. Their value is evaluated as implementation behavior rather than claimed as a new general algorithmic primitive.

The related work Exact Candidate Frontier Comparison for Decision Regression Testing of Precision-Constrained Selectors [29] is an unpublished manuscript supplied by the authors, who confirmed that status for this revision. It already introduces the observation contract, compact candidate-frontier comparison and checked proper-prefix attribution. Its labels/ID conventions, inherited observations and 255 counterfactual grid records are reused here. This extension studies selected-ID equivalence over policy domains and adds complete interval certificates, independent streaming consumption, explicit rational/binary64 semantics, executable conformance studies and verified domain-query interfaces. INHERITANCE_AND_PUBLICATION_BOUNDARY.md maps the reused and added components to hash-bound source snapshots. The earlier contract, witness machinery and reused data are not counted as new contributions.

1.4 Research Questions

RQ1 — Correctness: Can the method certify exact policy equivalence and divergent regions, including endpoint-only differences?

RQ2 — Testing effectiveness: How does complete policy checking compare with point-based baselines on controlled variants and documented upstream defects?

RQ3 — Computational cost: What are the costs of complete diagnostics and independent checking relative to matched baselines?

RQ4 — Practical policy queries: How do permitted policy domains and certificate reuse affect diagnostic outcomes and computational tradeoffs?

Section 2 defines the methods and the evidence collected for each question. Section 3 reports the four answers in the same order. Section 4 interprets their implications and limitations; Section 5 concludes.

2 Research Methodology

2.1 Problem Formulation and Selector Contract

Observation IDs are fixed, unique and aligned across versions. Labels are binary and fixed. Scores are finite. Each version has its own threshold candidates and counts, indexed by s in the displayed formula. Every observed score threshold induces a nonempty complete-group candidate; k denotes its positive count and n its selected size. Inclusive feasibility compares the rational policy to that candidate's precision. The displayed maximizer and membership use the same version index. Among feasible candidates choose the lexicographic maximum of positive count and selected size; abstain if none is feasible or the entire label vector has no positives. The policy domain A=[L,H] is specified with exact rational endpoints, 0<=L<H<=1. The inherited API excluded alpha=0; the extension defines its natural feasibility behavior explicitly.

$$j_s^*(\alpha) \;=\; \underset{j:\,\alpha n_{s,j} \le k_{s,j}}{arg\,max}\ (k_{s,j}, n_{s,j}),\;\; S_s(\alpha) \;=\; C_{s,j_s^*(\alpha)}$$

The output is an equivalence verdict over the declared domain, complete divergent regions with explicit boundary closure, and an optional observation-ID witness for each distinct divergent state. The original inputs and agreed contracts are required to check the record; selected counts alone do not establish selected-ID equality.

$$D_A \;=\; \{\alpha \in [L,H] : S_{old}(\alpha) \neq S_{new}(\alpha)\},\;\; PSC \;=\; \frac{length(D_A)}{H - L}$$

PSC is an exact uniform-domain length fraction, not a failure probability or risk measure. The Boolean equivalence verdict and endpoint evidence remain separate: the example with labels [1,0], scores [1,0] and domain [1/2,1] distinguishes inclusive from strict feasibility only at the two endpoints and has PSC = 0. Supplement S18 proves the relevant finite-domain obligations.

2.2 Exact Policy-Domain Verification

Finite partition theorem. Let B contain L, H and all candidate precision ratios of either version in (L,H). Both selected memberships are constant on each open interval between successive B values. Therefore equivalence throughout A holds if and only if memberships agree at each B value and one representative in each open interval.

Proof. Candidate feasibility is a comparison to its fixed ratio. Without a critical ratio inside an open interval, no feasibility predicate can change there. With fixed candidate priority and abstention, the selected membership is fixed. Singletons handle inclusive boundaries. These atoms partition the domain, proving necessity and sufficiency. Strict-feasibility mutants use the same critical values with different singleton outcomes. A midpoint alone is insufficient unless the critical set is complete.

Sort each version's scores, accumulate complete-group TP/N counts, and form exact reduced fractions. Sweep the union of critical policies in increasing order, evaluating every singleton and open midpoint. A priority heap removes candidates once they become infeasible; each candidate is removed at most once. Empirical precision need not be monotone along score prefixes. For reference selectors the largest feasible prefix is optimal because TP is nondecreasing with prefix size, but an explicit heap supports the reverse-secondary-order mutant too.

Given equally sized winning prefixes of size n, selected membership is equal exactly when the maximum new rank among old selected IDs is below n. Inverse ranks and prefix maxima make each equality check constant-time after preprocessing. For a divergent pair, a scan counts changed rows and supplies one observation ID whose old/new membership differs. Cache each distinct state pair and merge only adjacent identical states while preserving endpoint closure.

With G total score candidates and K critical values, core time is O(N log N+G log G+K) under unit-cost rational operations; witness scans add O(ND) for D distinct divergent state pairs. Auxiliary memory is O(N+G+K). Arbitrary-precision comparison has additional bit complexity. The independent verifier deliberately uses more expensive threshold-mask enumeration. No machine-checked proof or arbitrary black-box program verification is claimed.

Supplement S18 gives the core proof obligations for prefix-membership equality, producer complexity, streaming consumption, binary64 cuts and endpoint-only divergence. FORMAL_SPECIFICATION_AND_PROOFS.md provides additional implementation detail.

For a public binary64 policy interface, the critical set must also represent the rounding preimages of the actual feasibility guards. For an observed binary64 precision p, let q be its next larger binary64 value and let c be their exact rational midpoint. Under nearest-even conversion of the rational policy, the inclusive guard changes at c, with midpoint ownership determined by the even significand [26]. We form the union of these cuts, the exact-reference precision ratios and domain endpoints, then compare actual public outputs at every resulting breakpoint and midpoint. This admission argument requires policy-independent candidates and priority; hidden state or additional policy-dependent branches invalidate it. Supplement S18 supplies the guard-cut proof.

2.3 Certificate Generation and Independent Verification

The certificate records selector contracts, input SHA-256, domain, all critical fractions, state intervals, membership counts, an equality verdict, optional observation witness and exact PSC. It does not store all selected

IDs; the original input arrays are needed for verification. The independent verifier constructs actual threshold masks and integer bitsets, independently derives critical values, and checks coverage, winners, witnesses, counts and PSC. It shares the mathematical contract, but not the production heap or rank comparator.

The legacy consumer materializes all Boolean threshold masks. The streaming consumer instead imports neither the producer nor the legacy checker, counts one threshold mask at a time, and retains exact feasibility ratios, actual TP/N counts and candidate descriptors. A reverse suffix-maximum table over sorted ratios supplies fixed-priority winners by exact bisection. Current winning memberships are reconstructed and packed directly from the inputs; no all-candidate bitset cache or production rank-containment comparator is used.

The suffix winner argument is direct: feasible candidates form a suffix (lower bound for inclusive feasibility, upper bound for strict feasibility), and reverse induction computes that suffix's maximum fixed priority. Independent critical-point construction, ordered certificate coverage and endpoint ownership then justify checking all atom representatives. A forward cell cursor validates each probe; actual bitsets validate equality, witness IDs/memberships and changed-row counts. Exact divergent length and PSC are independently recomputed. Detailed lemmas and conditional soundness are in Supplement S18 and STREAMING_CHECKER_AND_FLOAT_CUT_PROOFS.md. These are mathematical arguments plus tests, not a machine-checked proof or independent human replication.

The consumer uses O(N+G+K+C) auxiliary descriptors/probes and current-mask storage, excluding arbitrary-precision operand sizes, for K critical values and C certificate cells. Direct counting and winner reconstruction retain worst-case O(NG) work; split-tie ordering adds O(N log N), with ratio/query/probe sorting costs stated in the proof supplement. In particular, G=N can still be quadratic. Suffix maxima, bisection and streaming are established techniques [17], not new general primitives.

A rational-domain query interface operates after independent certificate verification. Finite sets and decimal lattices return affected counts; intervals return exact intersected cells. All retain endpoint ownership and a rational witness when affected. Tampered inputs, out-of-domain requests and binary64 queries sent to the rational interface are refused. This checks the declared domain on the fixed inputs; it does not infer a production requirement.

We add VerifiedBooleanIndex. Its constructor copies and independently verifies the complete input-bound certificate, merges adjacent unequal cells only when their shared endpoint is included, and freezes a rational interval index with the checked input digest. A matching singleton hole between two open unequal regions is preserved. A query searches the ordered component endpoints and returns only Boolean existence plus one rational policy witness; it never constructs the full intersection list. Finite-set and decimal-lattice queries retain explicit rational semantics, domain validation and endpoint ownership. Counts and complete intersected regions remain available from the original API. This is an application of established interval union and binary search, not new parameter-region theory.

After construction, interval and singleton queries take O(log C) rational comparisons plus bounded candidate checks over C maximal components. A finite set performs point queries after validation/deduplication. A lattice may scan O(C) components before finding a lattice member or establishing absence, so it has no logarithmic worst-case claim. Construction and full independent checking remain separate costs; arbitrary-precision operands matter. BOOLEAN_QUERY_AND_ASSURANCE.md gives the reasoning and exact interface. Four targeted tests exercise zero-measure divergence, a matching singleton hole, input tamper/immutable reuse and unsupported domains.

`policy_cli.py` loads aligned NPZ arrays and writes a certificate after independent verification. `--fail-on-divergence` yields exit code 1 and a concrete interval/row witness. The pytest assertion wrapper reports the first divergent state; equality returns a certificate. An example GitHub Actions workflow runs tests and artifact verification locally configured for this folder. The workflow is supplied as a template; it has not been uploaded or executed on a remote CI service.

Supplement S5 documents unsupported-source fixtures, runtime refusals and CLI exit semantics. Refusal is never interpreted as equivalence.

2.4 Experimental Subjects and Datasets

Inherited evidence comprises all 50 retained comparisons (not only prior frontier-only positives), all 17 frozen synthetic fixtures and the public WDBC pilot. Their original data lineage is preserved. The 23 cases contributing the earlier 255 grid configurations are checked directly against those CSV records.

New pipelines comprise ten seeded splits for each of WDBC logistic + isotonic calibration, digits zero-vs-rest LinearSVC + sigmoid calibration, and generated imbalanced classification with random forest + score quantization. Fitting/calibration/evaluation indices are disjoint and exported; the RF reserves its unused calibration split. Test labels do not train or calibrate the model, but do define the empirical selector being tested. Ten splits are repetitions on one dataset family, not ten independent deployments. Small WDBC isotonic calibration is used as a structural workload; no calibration-quality advantage is claimed. Base numerical comparisons use float32 score casts, not naturally observed dependency releases.

Each newly fitted base input has eight selector/numerical variants, one row-ID fault and three valid controls. Variants change inclusive precision, threshold comparison, secondary priority, candidate omission, complete ties, feasibility counts, quantization or roundoff-scale scores. They are executable semantic fault models rather than a general source-code mutation engine. Full details appear in MUTATION_PROTOCOL.md.

The declared-semantics study uses configured finite contracts. A separate source-program study executes a reference selector and six changed programs on the same 30 fitted-pipeline inputs. The changed programs alter strict precision, score comparison, secondary priority, middle-group omission, tie splitting or the feasibility TP count. The adapter collects guard ratios from the running program and compares actual selected IDs with an unchanged direct-mask oracle. An explicit source-admission policy and AST checks restrict policy dependence to the instrumented guard, with repeated guard-event consistency checks. These restrictions support this audited local corpus; sampled trace agreement alone is not a completeness proof for arbitrary code. Supplement S2 documents the executable boundary and S5 the refusal cases.

SciPy release records identify the dependency-version context [13]; historical defect evidence is separately attributed to the issue and fix records.

The first documented upstream case is scikit-learn issue #23213 and fix #23214 [14,15], concerning precision-recall curve truncation at full recall. We replay unchanged affected/fixed function bodies through a counting compatibility bridge and compare their threshold families under the same downstream TP/N objective. Inputs comprise the original report protocol fitted in the current environment and the 30 pipeline workloads. Historical fitted coefficients and an old binary environment are not reproduced. Source hashes, commits, the excluded plotting sentinel and exact count checks are detailed in Supplement S19.

The second documented case is SciPy issue #6318 and fix #6351 [27,28], concerning overflow in nearest-interpolation midpoint construction on integer axes. Unchanged affected/fixed interp1d source is executed with a recorded compatibility namespace. Six prespecified axis fixtures include integer and floating controls; synthetic Unix-second carriers are also attached to the first 64 rows of each of the 30 existing score vectors. Interpolation is queried at observed knots under its identity requirement. These are constructed propagation fixtures, not independent time-series deployments. Supplement S19 retains commits, helpers and replay details.

A separate numerical-interface study executes the installed eval-toolkit TargetPrecisionSelector public API [22]. The complete unmodified 1.12.0 source is pinned at commit a6400e74c4534d6d79f17dee11ad903b6d664980 and distributed as a wheel with its license and metadata; Python 3.13.18 satisfies its requirement. No public method is patched or replaced by a custom scorer. The workload rule is the first 64 rows of every existing pipeline input, without filtering for positive outcomes. The wrapper converts rational policies to binary64 and maps only the documented no-feasible RuntimeError to abstention. The rational reference retains the maximum (TP,N) objective. This study concerns numerical contracts rather than a third documented defect.

Supplement S7 documents the executed TunedThresholdClassifierCV public-API adapter, its exact scorer and configuration refusals. It is another component of scikit-learn, not an additional independent project.

2.5 Baselines, Evaluation Metrics, and Experimental Protocols

For RQ1, we test the finite contract by exhaustive and randomized comparisons, independently replay certificates, check inherited grid outcomes and retain explicit endpoint/tamper cases. For RQ2, the measured outcomes are selected-ID divergence, executable nonconformance and baseline detections within each separately defined corpus. Snapshot probes use alpha = 0.50; the grid uses thirteen targets from 0.30 through 0.90. Corpus denominators and identity violations are never pooled with upstream defect counts.

Snapshot and grid tests observe actual configured mutant outputs. B4/current guard observe score arrays and do not inspect changed selector code. Their aggregate scores therefore have different scope. An exact certificate is configured with the mutant's declared finite contract; applying it to arbitrary real software requires a faithful adapter. Detection of the declared fault model does not certify that adapter automatically.

Correctness tests enumerate 4,680 binary-score/label pairs up to N=4 and 6,174 ternary-score cases up to N=3 (10,854 total), plus 1,000 seeded randomized variants. Additional pytest checks cover endpoint-only changes, invalid values/shapes, aligned identities, abstention, certificate tampering and actionable CI failures. These are finite implementation checks, not a proof of universal program correctness.

For RQ3, we measure elapsed time, within-task median/IQR and separately defined auxiliary memory. The traced producer protocol includes certificate construction and JSON serialization, ten repetitions per condition after warmup, randomized witness-mode order and one Windows host with Python 3.12.10/NumPy 2.3.5. Preallocated inputs are excluded from auxiliary MiB, which is not process RSS. The preparation-matched study in Supplement S3 and untraced ablation in S4 retain distinct instrumentation; their medians are not pooled with this protocol.

The complete-region comparator uses a direct Z3 encoding [16], not an independently optimized AllSMT implementation. Both routes must return maximal divergent components, exact endpoint ownership and PSC, a Boolean equivalence verdict and an existential policy/row witness per component. SMT expands each QF_LRA model to its complete feasibility-predicate cell and blocks that cell; residual UNSAT is required for completion. Unknown/timeout prevents certification. General predicate enumeration is established [23–25]; the comparison measures implementations under a common diagnostic output contract.

The smaller-candidate factorial crosses N in {1,000,10,000,100,000}, G in {10,30,90} and three omission patterns: none, one middle group or alternating groups. Scores form tied groups with gradually decreasing positive proportions and fixed-seed row permutations. All 27 tasks, including empty-divergence controls, are retained. Five repetitions per method/task randomize method order. Both routes regenerate counts/masks during timed preparation, serialize the diagnostic JSON and invoke the same region checker. Preparation, production/serialization, checking and total cost are recorded separately. Eight earlier external tasks remain a separate check. A larger-candidate extension uses N = 10,000, G in {250,500}, the same omission patterns and three randomized repetitions per method; these protocols are not pooled.

The paired experiment regenerates the same six N/G fixtures as the earlier checker study, prepares an identical certificate/payload outside timing, warms up both consumers, and randomizes their order within each of five repetitions. JSON parsing is recorded separately. A sixth invocation per consumer/group traces memory independently. The sixty untraced timing records and twelve separate memory records are never pooled. Each paired comparison uses identical inputs, policies, coverage, hash and witness obligations; these are not ratios against the differently instrumented producer benchmark.

For RQ4, we compare rational-reference and binary64-interface outcomes and intersect exact divergent components with declared policy sets. Quantiles use exact fractions and linear interpolation, with conversion only for display. Four rational decimal lattices on [0.30,0.90] are intersected algebraically using integer bounds and endpoint ownership. Representable-binary64 probes are embedded by their exact integer ratios, so they are not substituted for decimal targets. The 36 SciPy defect-study workloads are each assessed under eight predeclared finite, lattice and interval scenarios. Direct masks use cached executed interpolation outputs; an independent selected-bitset Z3 encoding [16] checks existence, with integer lattice indices or real interval policies. SAT witnesses are checked by direct selection; unknown/timeout aborts the comparison.

The reuse protocols distinguish two certificate APIs from direct-mask and incremental-Z3 request costs. Both certificate APIs pay construction, JSON roundtrip and the same independent full verification; the Boolean route also pays copying, merging, freezing and indexing. Direct masks and Z3 do not emit/check a complete diagnostic

certificate. Table 1 makes the evidence obligations explicit; checking a positive witness alone does not independently establish a direct negative result or an UNSAT proof.

See Table 1 for evidence-production and independent-checking obligations of the four Boolean query routes.

Table 1. Evidence-production and independent-checking obligations of the four Boolean query routes.

Route	Complete certificate produced	Independent full certificate check	Query operation used here
Full certificate API	Yes	Yes	Full API, then Boolean/witness projection
Boolean certificate API	Yes	Yes, with the same checker	Indexed Boolean/witness lookup
Direct masks	No persisted complete certificate	No certificate to check	Exact masks/critical-cell evaluation
Incremental Z3	No complete diagnostic certificate	No exported proof/certificate checked	SAT/UNSAT with rational SAT witness

The paired Boolean API study reuses six existing inputs, two request types, distinct rational point/window plans and batch sizes 1/8/32/128. One warmup per case/type/method precedes five fresh setups per batch/method, with seeded randomized order. Common interpolation, imports, disk I/O and human activity are excluded. Requests within a batch differ and no result memoization is used. Phase timings and total costs remain separate; medians are computed within each condition. The separate 720-record full-certificate request-cost protocol is retained separately in Supplement S14; it is not pooled with the paired API study.

This document revision used OpenAI Codex (OpenAI) for evidence-validation and visualization-authoring scripts. Figures are deterministic plots of the recorded research results; typography is adjusted without changing the underlying measurements. The research artifact also includes assistant-generated code and prose. Its computational scripts, raw measurements and independent checker outputs remain available; computational checks alone do not establish independent human review or replication.

3 Results

3.1 RQ1 Correctness and Exact Policy Equivalence

The recorded exact-archive execution passes 42 core pytest test items, including the exhaustive and randomized loops and the CRLF/tamper regression. The bundled results/artifact_verification.json is an older 41-test output snapshot; the execution count is recorded in the detached pytest log and regenerated results/pytest.xml. Application and mutation certificates are independently reverified against frozen inputs; the detached semantic reconciliation explains snapshot versus execution differences.

See Table 2 for fixed-input cohorts and policy-sensitive comparisons under their declared domains.

Table 2. Fixed-input cohorts and policy-sensitive comparisons under their declared domains.

Cohort	Pairs	Policy-sensitive pairs	Domain
Independent_pipeline	30	1	[0.30,0.90]
Public_WDBC	1	1	[0.30,0.90]
Retained	50	40	[0.30,0.80]
Synthetic	17	11	[0.30,0.80]

All 255 old grid outcomes and selected counts match. Among those 23 selected historical cases, exactly 15 are sensitive on their complete domains, the same case count found by the old grid; the extension supplies exact

intervals rather than discovering additional historical sensitive cases. It would be misleading to present the new method as uncovering missed cases in that subset.

For public WDBC, the precise divergent set is (43/51,9/10] on [3/10,9/10], giving divergent length 29/510 and PSC=29/306 (approximately 0.09477). The boundary 43/51 is excluded: equality still holds there. The old grid's two divergent values 0.85 and 0.90 are consistent with this interval.

Of 30 newly trained float32-cast comparisons, only `digits_svc_sigmoid_seed2` is policy-sensitive, with PSC=25/5084. This low frequency is reported as observed, without treating engineered mutations as naturally occurring numerical regressions.

The two independent consumers accept all 428 frozen application/mutation certificates under the same coverage, membership, witness and metric obligations, covering 41,348 probes. The additional finite tests exercise seven specifications and reject altered bindings, partitions, winners and metrics while retaining endpoint-only non-equivalence. The eval-toolkit suite contributes 18 passing upstream threshold tests separately from the 42 core test items. The independent consumer checks the finite-domain record; these results do not constitute a machine-checked proof or independent human replication.

Version outputs, numerical bridges and the distinction between policy differences and defects are detailed in Supplement S1.

The RQ1 evidence supports correctness of the implemented diagnostic obligations on the evaluated fixed inputs, including boundaries that PSC alone does not capture.

3.2 RQ2 Regression Detection and Upstream Defects

There are 360 runs: 240 selector/numerical variants, 30 row-identity violations, and 90 valid controls. Of the 240 variants, 183 are policy-divergent and 57 are equivalent on the tested input/domain. All 30 identity violations are separately rejected; PSC is not assigned to them. No exact policy alarm is raised for the 90 controls.

Figure 1 compares snapshot, thirteen-point grid and complete detection under the two distinct evaluation contracts.

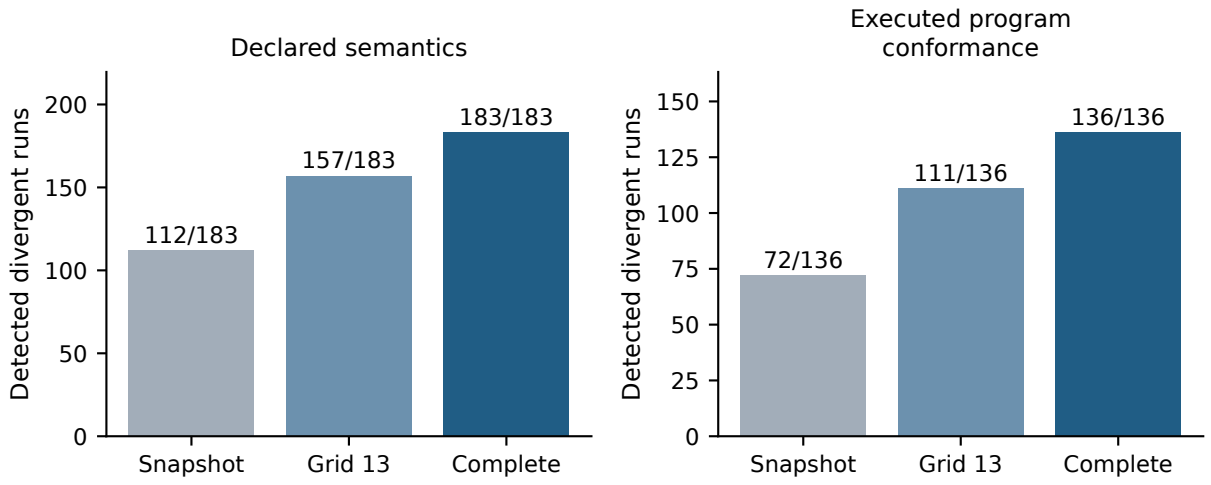


Figure 1 Detection under two distinct evaluation contracts. The denominators are not pooled.

See Table 3 for detection under declared selector semantics; the denominator excludes identity violations and valid controls.

Table 3. Detection under declared selector semantics; the denominator excludes identity violations and valid controls.

Test observation	Detected / 183 divergent mutants	Rate
Selected membership at alpha=0.50	112/183	61.2%
Thirteen-point output harness	157/183	85.8%
Score-only B4 structural flag (different scope)	47/183	25.7%
Current score guard at alpha=0.50 (different scope)	47/183	25.7%
Exact configured policy certificate	183/183	100.0%

The thirteen-point grid misses 26 divergent variants: 15 have positive-length divergent intervals and 11 differ only at critical points. Exact verification's 183/183 is agreement with an independent finite oracle for configured variants, not a general real-fault detection rate. Shared formal assumptions and known mutant semantics limit this evidence.

In the separate executable study, 136 of 210 program/input runs are nonconforming. Guard-complete checking detects all 136, compared with 111 for the grid and 72 for snapshot; all 30 reference controls conform. The unchanged oracle supplies the expected contract, rather than a supplied mutant specification. These are local source-program results, separate from the 183 configured divergent variants. Supplement S2 and Table S1 retain the program-level counts, and S12 retains operator-level declared-semantics records.

Across observed policy-equivalent variant/control runs, B4 raises 9 structural flags and the current guard raises 42 broad regression flags. These are not automatically errors: the tools have different observation contracts, and the current guard also observes numerical cutoff changes. We therefore avoid relabeling every frontier difference or threshold change as a policy defect. On selector-only faults with unchanged scores, the score tools lack the observation needed to see the modified code.

See Table 4 for detection of affected workloads in the two documented upstream defects; counts refer to workloads under the declared downstream contract, not independent bugs.

Table 4. Detection of affected workloads in the two documented upstream defects; counts refer to workloads under the declared downstream contract, not independent bugs.

Upstream defect	Workloads	Affected	Snapshot	Grid 13	Complete
scikit-learn #23213	31	15	14	15	15
SciPy #6318	36	31	26	31	31

The original report protocol reproduces a policy difference, with exact PSC 2059/5862 on [0.30,0.90]. Across all thirty-one workloads, fifteen exhibit policy effects; snapshot detects fourteen, while grid and full-domain checking both detect fifteen. Thus this external case establishes useful detection and localization of a documented external defect, but supplies no additional detected case over the thirteen-point grid. We report that null comparison directly. Sixteen workloads show no policy effect in the permitted domain despite the documented library defect. The fixed and current variants conform in all evaluated workloads. We do not pool these workloads with the internal mutant denominators or relabel the nine SciPy version differentials as confirmed bugs.

The 108 input-bound core certificates are independently replayed using actual interpolation outputs, exact membership checking and source/input bindings. Among the 31 affected workloads, PSC ranges from 0.055556 to 1.000000, markedly wider than the binary64 contract differences below. Five affected workloads are missed by snapshot; grid and full-domain both detect all 31. Thus incremental real-defect workload detection over grid remains zero; complete diagnostics add localization and replayable membership evidence. Thirty-four changed score vectors do not imply thirty-four policy regressions, and thirty-one divergent workloads remain one documented defect. Across studies, the confirmed-defect count is now two, in scikit-learn and SciPy; eval-toolkit

remains a separate contract-differential case. These projects share numerical dependencies and provide no multi-team deployment or real-bug detection-rate estimate.

RQ2 therefore separates a coverage advantage on controlled variants from a null incremental-detection result on the documented upstream cases. The latter support applicability and exact diagnosis, without establishing a broader real-bug detection-rate advantage.

3.3 RQ3 Computational and Verification Costs

In the traced producer protocol, the N = 100,000 divergent witness-enabled condition has median 528.06 ms (IQR 40.73) and emits 111 witness-bearing state cells. Its witness-disabled counterpart has median 520.59 ms (IQR 120.04). These are within-condition descriptive measurements, not a direct latency ratio against another instrumentation protocol. All producer conditions remain in Supplement Table S9; the phase ablation and preparation-matched costs remain in S3–S4.

Across the smaller-candidate factorial, the two routes produce agreeing complete diagnostics and all 54 saved certificates are independently replayed. The five-repetition protocol yields 270 timing records. Partition has the smaller total median on 25 of 27 tasks; the remaining two have an equal or smaller SMT median. Table 5 shows the N = 100,000 slice; the raw factorial retains the other scales and unfavourable cases.

See Table 5 for complete diagnostic comparison at N = 100,000 over the smaller candidate-count factorial; totals include independent checking.

Table 5. Complete diagnostic comparison at N = 100,000 over the smaller candidate-count factorial; totals include independent checking.

G / omission	Critical points	Components	Partition total ms	SMT total ms
10 / alternating	10	5	59.26	78.39
10 / none	10	0	52.90	65.71
10 / one	10	1	58.68	64.17
30 / alternating	27	13	111.01	193.82
30 / none	27	0	100.06	109.05
30 / one	27	1	103.76	129.67
90 / alternating	79	39	494.01	1058.73
90 / none	79	0	323.85	317.51
90 / one	79	1	329.47	391.97

The larger-candidate extension produces 36 timing records and twelve independently replayed certificates, again with identical complete diagnostics. Its tasks reach 430 critical points and 215 divergent components. Supplement Table S10 retains all six larger-candidate tasks, including cases where SMT has the lower total median. Three repetitions are descriptive and are not combined with the smaller-candidate five-repetition study.

Figure 2 compares paired independent-consumer timing and separately measured auxiliary memory on the same fixtures.

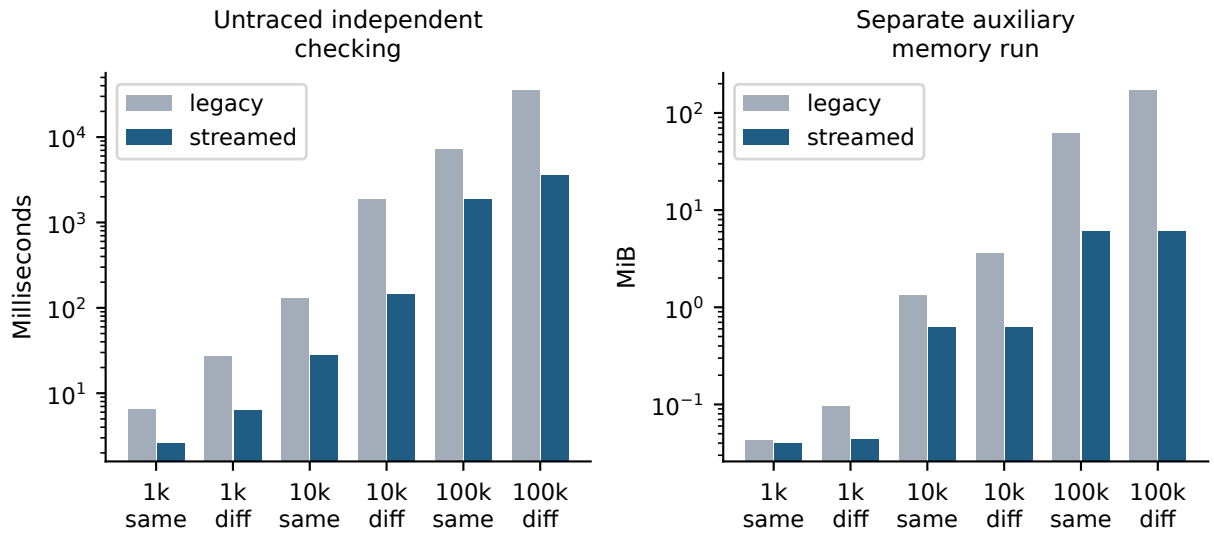


Figure 2 Paired independent consumer medians and separately traced auxiliary memory peaks. Both axes use logarithmic scales.

See Table 6 for paired independent-consumer timing and separately traced auxiliary memory on identical fixtures.

Table 6. Paired independent-consumer timing and separately traced auxiliary memory on identical fixtures.

N / fixture	Legacy median (IQR), ms	Streaming median (IQR), ms	Legacy peak MiB	Streaming peak MiB
1,000 / invariant	6.41 (0.67)	2.56 (0.15)	0.04	0.04
1,000 / divergent	26.98 (1.38)	6.24 (0.46)	0.09	0.04
10,000 / invariant	130.23 (12.14)	27.72 (6.19)	1.33	0.63
10,000 / divergent	1859.66 (59.43)	145.62 (22.20)	3.56	0.63
100,000 / invariant	7097.14 (759.47)	1843.19 (48.92)	61.42	6.00
100,000 / divergent	35034.92 (360.42)	3557.72 (139.89)	169.85	6.00

Memory peaks are auxiliary allocations with inputs/certificate prepared in advance and one traced run per group. Five timings on one host do not establish a universal CI budget. The implementation preserves a deliberately different verification route while reducing retained masks; shared dependencies and mathematical assumptions remain trusted. Results show an implementation tradeoff under this contract, not a general theorem that checking is always cheaper than production.

RQ3 shows implementation tradeoffs under specified diagnostic obligations. Neither the partition comparison against this Z3 encoding nor the streaming comparison against the legacy consumer establishes general speed superiority, an asymptotic improvement in all work, or a production latency guarantee.

3.4 RQ4 Allowed-Domain Queries and Certificate Reuse

Across thirty workloads, 884 critical-point/midpoint policies and 1500 public invocations (including the grid/snapshot baselines) are executed. The guard model matches every actual output. Twenty-seven workloads have narrow differences from the exact rational reference; neither snapshot nor the thirteen-point grid detects any of them. These are twenty-seven semantic differentials, not twenty-seven bugs or regressions between upstream releases. The library documents a floating policy interface, so the rational reference is a caller-selected

stronger/different contract, not an upstream requirement it is shown to violate. This numerical-contract case adds no confirmed defect; on the documented scikit-learn defect, incremental detection over grid remains zero.

Each input-bound certificate preserves exact rounding-cut fractions, ownership, actual/reference bitsets, row witnesses, complete divergent regions and PSC. The validator calls the installed selector again at all 884 policies and checks every stored field. We additionally probe actual representable binary64 precision policies: 196 such probes expose differences on 19 workloads. Grid/snapshot targets are exact rational fractions, whereas this additional group embeds each binary64 value by its exact integer ratio; it does not reinterpret a decimal grid target as its binary approximation. Thus the continuous-domain finding is not automatically equated to a difference at a policy the float API can represent. With labels [1,1,0,0,0] ordered by decreasing distinct score, the representable float 0.4 exceeds exact 2/5: at the exact rational embedding of that float, the float selector admits all five while the rational reference admits four. At exact decimal 2/5 both admit five. This is a designed explanation of distinct policy contracts, not a newly documented upstream bug. Tiny interval length is not a deployment probability or impact estimate. Adoption should decide whether binary64 semantics are acceptable, rather than treating every differential as a release-blocking defect.

Figure 3 shows the positive-PSC distribution and the effect of the declared policy scope on observed differences.

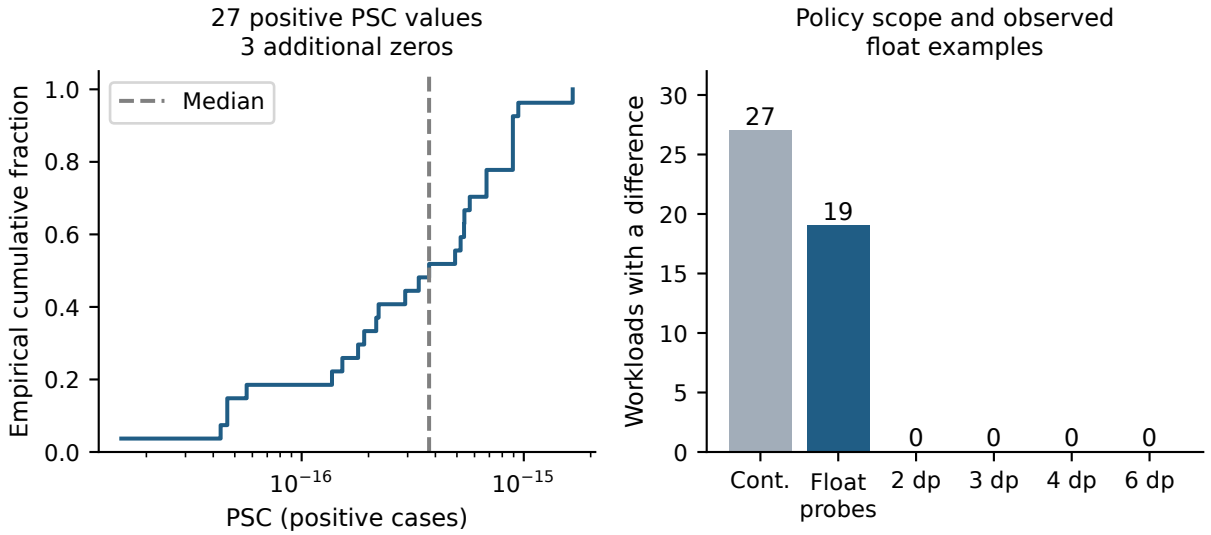


Figure 3 Positive PSC distribution and policy scope. Float probes count observed representable examples; decimal bars use exact complete lattice intersections. These are not deployment probabilities or common-domain detection rates.

The 27 positive-PSC workloads have median PSC 3.747190e-16, with minimum 1.541976e-17 and maximum 1.660140e-15; three additional workloads have zero PSC. No affected policy occurs on any of the four declared decimal lattices for these 30 workloads. Supplement Tables S11–S12 retain the full conditional distribution and lattice counts. The separate 196 representable-binary64 probes expose differences on 19 workloads under their different exact embedding. These observations describe numerical-interface contracts rather than deployment probabilities.

At individual verified policies, membership can still change: across these fixtures the maximum changed-ID count is 41 of 64. The largest precision shortfall observed at stored probes is 1.048544e-16; separately, the exact supremum across all covered cells is 1.048544e-16. That supremum includes limits at open cell endpoints and need not be attained. These quantities do not turn a tiny arithmetic discrepancy into demonstrated harm. The observed labels provide no deployment-frequency, utility, monetary loss or human-outcome data.

The same exact permitted-set assessment is applied separately to the SciPy defect fixtures: at two decimal places, 31 of 36 workloads have affected targets, comprising 1,611 of 2,196 allowed policy/workload pairs. Per-case counts for all four resolutions are retained in scipy_allowed_policy_counts.csv. These are constructed-fixture coverage counts, not observed deployment frequencies or independent defect counts.

See Table 7 for verified certificate reuse for eight declared policy-domain scenarios on the same 36 workloads.

Table 7. Verified certificate reuse for eight declared policy-domain scenarios on the same 36 workloads.

Allowed policy scenario	Policies per workload	Workloads with an affected target	Affected policy/workload pairs
Thirteen-point grid	13	31 / 36	340
Declared set {0.50,0.80,0.90}	3	30 / 36	86
Decimal lattice 2 places	61	31 / 36	1,611
Decimal lattice 3 places	601	31 / 36	15,886
Decimal lattice 4 places	6,001	31 / 36	158,657
Decimal lattice 6 places	600,001	31 / 36	15,863,561
Closed interval [0.30,0.90]	Continuous	31 / 36	Not a finite count
Closed interval [0.80,0.90]	Continuous	30 / 36	Not a finite count

All 288 scenario/workload query answers agree with the independently constructed Z3 encoding [16], with no unknown outcome. Exhaustive checking of finite sets and the two- and three-place lattices covers 24,408 policies and agrees on affected counts; finer lattices use exact algebraic intersections. For pipeline_axis_generated_rf_quantized_seed7, the continuous interval is affected but the declared three-element set is not. This is a change in the scope of the allowed-domain verdict on a study scenario, not a newly discovered defect or an elicited production requirement.

The paired API protocol yields 960 timing records and 192 median/IQR groups. Its 2,028 existing case/query plans produce 40,560 repeated method answers across four routes and five repetitions; all agree and saved rational witnesses are independently checked. These repetitions do not enlarge the empirical defect corpus. Table 8 reports batch-128 total medians, including each route's setup; phase medians, IQRs and other batches remain in the raw records.

See Table 8 for batch-128 total median query cost in milliseconds, including route-specific setup, on the existing six inputs.

Table 8. Batch-128 total median query cost in milliseconds, including route-specific setup, on the existing six inputs.

Workload	Request	Full certificate	Boolean certificate	Direct masks	Incremental Z3
Upstream int8	Point	3.41	2.62	1.75	45.44
Upstream int8	Window	11.23	6.03	10.50	74.22
Unix int32	Point	7.16	4.89	2.06	91.57
Unix int32	Window	7.84	2.61	7.63	56.65
Small int32 control	Point	3.59	3.68	1.95	32.11
Small int32 control	Window	5.52	3.26	9.53	24.25
WDBC seed 0	Point	15.01	7.59	2.77	111.84
WDBC seed 0	Window	27.29	5.70	11.96	84.00
Digits seed 0	Point	43.92	10.80	19.31	189.20
Digits seed 0	Window	177.87	16.38	32.18	164.59
RF seed 0	Point	11.40	6.01	6.05	163.68
RF seed 0	Window	21.06	5.93	10.75	129.52

In the assurance-matched panel, Boolean lookup lowers the query-phase median in 12/12 batch-128 combinations, by 2.06–35.39 times. Including all setup, its total median is lower in 11/12 combinations; the remaining cases are retained. Thus part of the full-certificate route's cost came from its output construction, while construction and independent certificate checking still have a price. These are descriptive paired medians on one host, not a statistical significance result or a universal speed guarantee.

In the four-route request-cost panel, the Boolean route has lower total medians than direct masks for 2/6 point and 6/6 window inputs, and than this incremental Z3 encoding for 6/6 and 6/6. Those comparisons have the unequal obligations shown above and are not a ranking of equal verification guarantees or optimized external tools. Direct checking remains a valid simpler choice for a small finite allowed set. Computational reuse results do not measure developer debugging time, production policy requirements, institutional adoption or deployment break-even. The contribution remains exact diagnosis and checkable evidence with explicit interface and cost boundaries.

RQ4 demonstrates that declared domains can change the diagnostic answer and that API output work affects measured reuse costs. It does not establish which domains users require or a general break-even point for creating certificates.

4 Discussion

4.1 Interpretation of the Main Findings

The four answers distinguish completeness, observed detection and the usefulness of a diagnostic record. RQ1 establishes agreement with the finite empirical contract on the evaluated inputs, including endpoint-only divergence. RQ2 shows why that completeness must not be translated into an undifferentiated detection claim: controlled variants expose gaps in point coverage, but the two documented upstream defects add no affected workload over the thirteen-point grid. In those cases, the contribution is the complete location of the change, correct boundary ownership and a record another implementation can check.

A certificate answers more than the existence question observed by one positive test. It records which policies are affected and binds that statement to input identities, selector semantics and independently checked memberships. That makes it a concrete object for inspection and handoff, conditional on the admitted contract. This study has not measured whether developers debug faster, whether teams adopt the record, or whether a release decision improves. Computational replay supports the artifact's checkability; it does not substitute for those human or operational outcomes.

RQ3 and RQ4 also limit the cost argument. The streaming consumer reduces retained mask materialization relative to the legacy route, while worst-case threshold/witness work remains substantial. The Boolean index removes unnecessary full-intersection output for its query task, yet its constructor still pays complete construction and independent verification. Direct masks remain cheaper in several finite-query cases. A requirement for persistent, independently checked complete diagnostics must therefore be established before interpreting certificate setup as justified work.

4.2 Comparison with Existing Verification and Testing Approaches

Snapshot and grid testing observe the executable at specified policies, which is a useful and sometimes sufficient release test. If every allowed policy belongs to an explicitly finite tested set, exhaustive direct testing is already complete for that set. For an interval, completeness instead depends on deriving all behavior-changing cuts under an admitted contract. The diagnostic method provides that derivation and preserves isolated boundaries; it does not invalidate finite testing or imply that every application needs an interval certificate.

General SMT can express the selected-ID questions, as the direct Z3 comparisons demonstrate [16]. Predicate enumeration and AllSMT already explore regions [23–25], while parameter-synthesis tools address their own model classes [6,7,20]. Our complete-region cost study matches output obligations for a particular nested selector encoding; the four-route query study deliberately does not equate certificate assurance with direct-mask or solver request cost. None of these experiments is a reproduction of an independently optimized general AllSMT tool. Supplement S22 and Table S13 retain the detailed comparison and the boundary between empirical and conceptual evidence.

The producer/checker architecture follows certifying-computation foundations [9,17] and is related to proof-carrying approaches [10]. Here, certificates are finite membership evidence consumed by Python implementations sharing a mathematical contract, not machine-checked proofs of arbitrary code. Likewise, robust classification [18], parametric monitoring [19] and metamorphic testing [4,11,12] concern related but different objects. The specialization should be evaluated by the precision of its contract and evidence, rather than by claiming that those foundations cannot represent complete regions.

4.3 Implications for Regression Testing and Release Assurance

A team using this workflow would first declare observation identities, selector semantics, numerical-interface conversion and the policies that a release must support. A CI result can then distinguish checked equivalence, observed divergence with a witness, and refusal because assumptions are unsupported. Refusal must not be interpreted as compatibility. The supplied CLI and local CI executions demonstrate this separation; the workflow template is not evidence of a remote deployment, a suitable testing frequency or an operating service-level objective.

The permitted-domain results illustrate why an explicit requirement matters. A finite allowed set can be unaffected even when the continuous domain diverges, and zero PSC can still hide an affected singleton. A release assessment should preserve both facts rather than erase small regions with a tolerance or label every numerical-interface difference a bug. The eval-toolkit study is especially restrictive: its rational reference is a caller-selected contract, so the observed binary64 differences do not by themselves establish an upstream violation.

Certificate reuse is most defensible as a means of retaining exact scope and checking subsequent domain questions against the same input-bound diagnostic object. Its practical value still depends on the need for that object, input stability, query volume and the costs of construction, verification and storage. A small fixed policy set can favour direct masks. Prospective deployment studies, independent developer tasks and an expanded documented-defect corpus would be needed to test broader engineering benefits; the present results do not estimate them.

4.4 Threats to Validity and Limitations

Internal and construct validity. The independently coded checker shares the written contract with the producer and may share conceptual errors; finite tests are not a machine-checked proof. Input hashes protect accidental binding integrity, not source authenticity or label quality. PSC measures interval length, excludes isolated points and is not a deployment probability. The declared-semantics study supplies mutant contracts; the program-conformance study instead uses an unchanged reference, with source admission and guard-coverage assumptions limiting its scope.

External validity. Historical cases are related, generated fixtures are controlled, and pipeline seeds repeat three local dataset families. The two documented defects are historical-source replays through compatibility bridges with constructed propagation carriers and shared dependencies. Grid13 detects every affected workload in both cases, so no added real-bug detection rate is established. Results concern fixed labeled inputs, not predictive generalization. There is no independent developer triage study, production-policy inventory or prospective deployment.

Performance validity. Results come from one host and modest repetitions; allocation tracing and untraced timings are separate protocols. Input/candidate structure and arbitrary-precision arithmetic matter, and checking/witness work can remain O(NG)/O(ND). The paired certificate APIs share full construction/checking obligations; four-route request costs do not share evidence-production obligations. Reported medians and speedups remain descriptive for each workload and protocol.

Contribution and overlap. Region partitioning, constrained selection, AllSMT and certificate checking have established foundations. The 28-source check is focused, not systematic, and does not prove absence of closer work. The related unpublished manuscript [29] is disclosed with its reused components; the current integration and evaluation do not establish general algorithmic originality or industrial effectiveness.

5 Conclusion

This study develops exact policy-aware regression diagnosis for fixed observations and precision-constrained threshold selectors. The finite contract yields complete divergent regions, endpoint-correct selected-ID comparisons and input-bound certificates with independent consumption. The evaluation supports correctness on the admitted inputs, stronger coverage than point tests on controlled variants, and detailed diagnosis of two documented upstream defects. Those two cases show no incremental affected-workload detection over the thirteen-point grid.

Streaming consumption and the Boolean certificate API expose implementation costs and avoid some unnecessary materialization, while complete checking and setup remain real obligations. Permitted-domain queries refine the scope of the diagnosis but do not establish production-policy relevance. The contribution is a specialized auditable workflow built on established verification foundations; general algorithmic novelty, developer productivity and industrial effectiveness remain outside the demonstrated evidence.

Reproducibility and Data Availability

Frozen inputs, split manifests, source, tests, raw timing records and certificates accompany this submission in Reproducibility_Artifact.zip (SHA-256 bf4ad2b56d9d3ba836494674e3493e75966c8e28763c6ac5bcf6b40954d36591). Reproducibility_Evidence.zip supplies the detached execution records and regenerated-output reconciliation. For reviewer access, the submission package supplies both archives alongside this manuscript and the supplementary material. Author-owned code and third-party data/dependencies retain their respective accompanying licenses. Newly trained pipeline arrays derive from the bundled WDBC snapshot, scikit-learn digits and generated data.

Supplement S9 details the historical standalone repair, frozen-input execution, optional retraining and the limits of each assurance layer.

The standalone artifact contains the code, frozen inputs, dependency specifications and source-integrity bindings needed by its 33-stage default entry point. It includes requirements files and the eval-toolkit wheel, rather than a complete bundled Python environment; other required packages must be installed as specified. Files named after earlier revisions implement cumulative experiments and audits within this single artifact; reproducing the study does not require executing earlier release ZIPs, installing earlier project versions, or accessing a neighboring submission directory. Default execution reuses frozen scores; optional retraining and historical dependency-binary execution are distinct procedures with additional requirements.

The exact final artifact is frozen before execution; its detached execution report binds its SHA-256 and records every stage exit status. The report is distributed alongside the artifact, avoiding a self-referential archive hash. A local Windows execution establishes neither native Linux validation nor independent institutional reproduction. The reading copy uses the recorded measurement snapshot; rerun results retain their own timing records. See REPRODUCIBILITY_V11.md for commands, scope and release-evidence locations.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

OpenAI Codex (OpenAI) assisted with manuscript organization, language revision, mathematical presentation, document formatting and authoring scripts. The associated research artifact also contains assistant-generated code and prose. Responsibility for evaluating and approving the scientific claims, software, references and final publication remains with the human authors.

References

[1] Rath, P.; Hughes, M. (2022). Optimizing Early Warning Classifiers to Control False Alarms via a Minimum Precision Constraint. PMLR 151, 4895–4914. https://proceedings.mlr.press/v151/rath22a.html

[2] Mackey, A.; Luo, X.; Eban, E. (2018). Constrained Classification and Ranking via Quantiles. https://arxiv.org/abs/1803.00067

[3] Tasche, D. (2018). A plug-in approach to maximising precision at the top and recall at the top. https://arxiv.org/abs/1804.03077

[4] Duque-Torres, A.; Klammer, C.; Fischer, S.; Pfahl, D.; Ramler, R. (2025). Assessing the strength of Metamorphic Testing applied to optimisation software—Experience from industry. Information and Software Technology, 186, 107807. https://doi.org/10.1016/j.infsof.2025.107807

[5] scikit-learn developers. CalibratedClassifierCV, version 1.9.1 documentation. https://scikit-learn.org/stable/modules/generated/sklearn.calibration.CalibratedClassifierCV.html (accessed 30 October 2026).

[6] Storm project. Running Storm on Parametric Models. Official documentation. https://www.stormchecker.org/documentation/usage/running-storm-on-parametric-models (accessed 30 October 2026).

[7] Junges, S.; Ábrahám, E.; Hensel, C.; Jansen, N.; Katoen, J.-P.; Quatmann, T.; Volk, M. (2024). Parameter synthesis for Markov models: covering the parameter space. Formal Methods in System Design, 62, 181–259. https://doi.org/10.1007/s10703-023-00442-x

[8] Godlin, B.; Strichman, O. (2009). Regression Verification. DAC. Author manuscript: https://ofers.dds.technion.ac.il/publications/dac09.pdf

[9] Alkassar, E.; Böhme, S.; Mehlhorn, K.; Rizkallah, C. (2013). A Framework for the Verification of Certifying Computations. https://arxiv.org/abs/1301.7462

[10] Necula, G. C.; Schneck, R. R. (2002). Proof-Carrying Code with Untrusted Proof Rules. International Symposium on Software Security. Author manuscript and formalization: https://people.eecs.berkeley.edu/~necula/ISSS02/

[11] Yang, Y.; Li, Z.; Wang, H.; Xu, C.; Ma, X. (2021). Towards effective metamorphic testing by algorithm stability for linear classification programs. Journal of Systems and Software, 180, 111012. https://doi.org/10.1016/j.jss.2021.111012

[12] Xie, X.; Ho, J. W. K.; Murphy, C.; Kaiser, G.; Xu, B.; Chen, T. Y. (2011). Testing and validating machine learning classifiers by metamorphic testing. Journal of Systems and Software, 84(4), 544–558. https://doi.org/10.1016/j.jss.2010.11.920

[13] SciPy project. Release news and versions 1.15.3 and 1.18.1. https://scipy.org/news/

[14] scikit-learn issue #23213 (2022). precision_recall_curve is not returning the full curve at high recall. Upstream bug report and maintainer discussion. https://github.com/scikit-learn/scikit-learn/issues/23213

[15] scikit-learn PR #23214 (2022). FIX compute precision-recall at 100% recall. Merged 2 May 2022. https://github.com/scikit-learn/scikit-learn/pull/23214

[16] de Moura, L.; Bjørner, N. (2008). Z3: an efficient SMT solver. TACAS. https://www.microsoft.com/en-us/research/publication/z3-an-efficient-smt-solver/

[17] McConnell, R. M.; Mehlhorn, K.; Näher, S.; Schweitzer, P. (2011). Certifying algorithms. Computer Science Review, 5(2), 119–161. https://doi.org/10.1016/j.cosrev.2010.09.009

[18] Provost, F.; Fawcett, T. (2001). Robust Classification for Imprecise Environments. Machine Learning, 42, 203–231. https://arxiv.org/abs/cs/0009007

[19] Chen, F.; Roşu, G. (2009). Parametric trace slicing and monitoring. TACAS, 246–261. https://experts.illinois.edu/en/publications/parametric-trace-slicing-and-monitoring/

[20] Dehnert, C.; Junges, S.; Jansen, N.; Corzilius, F.; Volk, M.; Bruintjes, H.; Katoen, J.-P.; Ábrahám, E. (2015). PROPhESY: A PRObabilistic ParamEter SYnthesis Tool. CAV, LNCS 9206, 214–231. https://www.cs.ru.nl/personal/nilsjansen/files/publications/dehnert-et-al-cav-2015.pdf

[21] scikit-learn developers. TunedThresholdClassifierCV API: callable scorer, explicit threshold arrays and prefit configuration. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TunedThresholdClassifierCV.html (accessed 30 October 2026).

[22] Behring, B. eval-toolkit: threshold selection methodology and source. https://brandon-behring.github.io/eval-toolkit/methodology/thresholds.html ; pinned packaging/source at https://github.com/brandon-behring/eval-toolkit/tree/a6400e74c4534d6d79f17dee11ad903b6d664980 (accessed 30 October 2026).

[23] MathSAT developers. API reference: msat_all_sat, enumeration over important atoms. https://mathsat.fbk.eu/apireference.html (accessed 30 October 2026). Used to contextualize predicate enumeration, not as an evaluated implementation.

[24] Morettin, P.; Passerini, A.; Sebastiani, R. (2017). Efficient Weighted Model Integration via SMT-Based Predicate Abstraction. IJCAI, 720–728. https://www.ijcai.org/proceedings/2017/100

[25] Spallitta, G.; Sebastiani, R.; Biere, A. (2025). Disjoint projected enumeration for SAT and SMT without blocking clauses. Artificial Intelligence, 345, 104346. https://doi.org/10.1016/j.artint.2025.104346

[26] Andrlon, M. (2023). Finding Normal Binary Floating-Point Factors Efficiently. Journal of Automated Reasoning, 67, article 11. https://doi.org/10.1007/s10817-023-09659-1

[27] SciPy issue #6318 (2016). Scipy interp1d nearest not working for high values on x-axis. Upstream defect report. https://github.com/scipy/scipy/issues/6318

[28] SciPy PR #6351 (2016). BUG: #6318 Interp1d nearest integer x-axis overflow issue fixed. Merged 6 July 2016. https://github.com/scipy/scipy/pull/6351

[29] Exact Candidate Frontier Comparison for Decision Regression Testing of Precision-Constrained Selectors. (2026). Unpublished related manuscript supplied by the authors; status confirmed by the authors for this revision. Source snapshot: provenance/q2_reference/main.tex.